

Guião Técnico

AgroConecta

Como o programa funciona por dentro — front-end e back-end,
explicado para quem já programa em Java.

Plataforma agrícola para Moçambique · React + TypeScript · Supabase · Cloudflare

1. Antes de mais: a ideia toda

O **AgroConecta** é uma aplicação **web** (corre no browser, não é instalada) que liga agricultores, donos de terreno e compradores em Moçambique: um marketplace de terrenos e produtos, gestão de cultivos, negociações com chat, um assistente agrícola com IA e planos pagos.

Uma aplicação web moderna divide-se em duas metades. Guarda estes dois nomes, porque tudo o resto encaixa neles:

- **Front-end** — tudo o que o utilizador **vê e toca** (botões, páginas, formulários). Corre no **browser** do utilizador.
- **Back-end** — tudo o que acontece **nos bastidores**: guardar dados, autenticar utilizadores, guardar fotos, processar pagamentos. Corre em **servidores**, longe do utilizador.

ANALOGIA COM JAVA — a divisão front / back

Se já fizeste um projeto **Spring Boot** com uma interface: o back-end é o teu Spring (controladores, serviços, base de dados); o front-end seria o Swing/JavaFX ou o HTML que o utilizador vê. Aqui é a mesma separação, só que o front-end é muito mais rico e vive no browser.

2. Vista de cima — as peças e como falam

O AgroConecta não tem **um** servidor gigante. Tem o front-end no browser a falar com dois serviços especializados. Este é o mapa mental mais importante do guião:

Peça	O que é	Papel no AgroConecta
Browser (React)	O front-end, no telemóvel/PC do utilizador	Mostra as páginas e reage aos cliques
Supabase	Base de dados + login + fotos + tempo real	Guarda e devolve os dados; autentica
Cloudflare Functions	Pequenas funções no servidor	Escondem as chaves secretas (IA e pagamentos)
Cloudflare Pages	Alojamento do site	Serve os ficheiros do front-end ao mundo

O detalhe elegante: o browser fala **diretamente** com o Supabase para ler e escrever dados (protegido por regras de segurança, que já vamos ver). Só a IA e os pagamentos passam pelas Cloudflare Functions — porque essas usam **chaves secretas** que nunca podem chegar ao browser.

ANALOGIA COM JAVA — Supabase

Pensa no Supabase como um **PostgreSQL + Spring Security + serviço de ficheiros**, tudo pronto e gerido por eles. Não escreveste o servidor de base de dados nem o de login: usas os deles através de uma biblioteca (o cliente supabase-jts).

3. As linguagens (e o teu Java como referência)

Usaste sobretudo **TypeScript** com **React**. Aqui está o mapa direto a partir do que já conheces:

No AgroConecta	Equivalente mental em Java	Para que serve
TypeScript	Java (com tipos), mas mais solto	A linguagem de toda a lógica
React (JSX)	JavaFX/Swing, mas declarativo	Construir a interface
Tailwind CSS	CSS por 'classes utilitárias'	O aspeto visual (cores, espaços)
Vite	Maven / Gradle	Compilar e empacotar o projeto
npm	Maven Central / dependências	Instalar bibliotecas
SQL (Postgres)	SQL + JPA/Hibernate	Falar com a base de dados

TypeScript é JavaScript com **tipos**. Se sabes Java, já pensas em tipos — a diferença é que aqui os tipos são uma camada opcional por cima de uma linguagem dinâmica. Exemplo:

```
// TypeScript
type Plano = { cultura: string; area: number };

function resumo(p: Plano): string {
  return p.cultura + ' - ' + p.area + ' m2';
}
```

ANALOGIA COM JAVA — os tipos

`type Plano = {...}` é como uma **classe/record** só com campos. `function resumo(p: Plano): string` lê-se exatamente como em Java: recebe um `Plano`, devolve uma `String`. A diferença: em TypeScript os tipos desaparecem quando o código corre — servem só para te avisar de erros enquanto escreves.

4. O front-end — o que o utilizador vê

4.1 Componentes: a peça central do React

Toda a interface é feita de **componentes**. Um componente é uma **função que devolve o que aparece no ecrã**. Podes encaixá-los uns nos outros como peças de Lego (um botão dentro de um cartão, dentro de uma página).

```
function BotaoEntrar() {  
  return <button>Entrar</button>;  
}
```

Aquele `<button>` dentro do código é **JSX** — HTML escrito dentro do TypeScript. Parece estranho ao início, mas é só uma forma de descrever a interface.

ANALOGIA COM JAVA — componentes

Um componente é como uma **classe pequena com um único método que devolve UI**. Onde em JavaFX farias `new Button("Entrar")` e o adicionarias a um painel, aqui a função **devolve** a descrição do botão e o React trata de o desenhar.

4.2 Props: passar informação a um componente

Um componente recebe dados de fora através de **props** (propriedades) — como argumentos de um método.

```
function Saudacao({ nome }: { nome: string }) {  
  return <p>Ola, {nome}!</p>;  
}  
  
// usar: <Saudacao nome="Shelven" />
```

ANALOGIA COM JAVA — props

Props = os parâmetros do método. `<Saudacao nome="Shelven" />` é o mesmo que chamar `saudacao("Shelven")`. São só de leitura: o componente não altera as props que recebe.

4.3 Estado (useState): memória que redesenha o ecrã

Se um componente precisa de **lembrar** algo que muda (o texto escrito, se um menu está aberto), usa **estado**. Quando o estado muda, o React **redesenha automaticamente** essa parte do ecrã.

```
const [conta, setConta] = useState(0);  
  
<button onClick={() => setConta(conta + 1)}>  
  Cliquei {conta} vezes  
</button>
```

ANALOGIA COM JAVA — estado

Em Swing, mudavas um atributo e depois chamavas `repaint()` à mão. Aqui, chamar `setConta(...)` muda o valor e manda redesenhar sozinho. O estado é um atributo de instância que, ao mudar, atualiza a interface automaticamente.

4.4 useEffect: reagir a acontecimentos

Às vezes é preciso fazer algo **quando o componente aparece** ou quando um valor muda — por exemplo, ir buscar dados ao servidor. Isso faz-se com `useEffect`.

ANALOGIA COM JAVA — useEffect

É como o **ciclo de vida** de um componente (o teu `init()` / `onStart()`). 'Quando isto aparecer no ecrã, corre este código.'

4.5 As outras ferramentas do front-end

- **Tailwind CSS** — em vez de escrever CSS à parte, pões classes no elemento: `className="rounded-lg bg-primary"` (cantos redondos, fundo verde). Rápido e consistente.
- **react-router** — decide que página mostrar conforme o endereço (`/marketplace`, `/perfil`). É a 'navegação' da app.
- **React Query** — trata de ir buscar dados ao servidor, guardá-los em cache e recarregá-los. Poupa muito código.
- **shadcn/ui + Radix** — componentes prontos e acessíveis (botões, caixas, menus) que só afinamos.

4.6 Como o front-end está organizado

O código está arrumado por **funcionalidade** (feature), não por tipo de ficheiro. Cada pasta em `src/features/` é uma área da app, com as suas páginas, componentes e serviços de dados:

```
src/features/
  auth/          -> login e sessao
  marketplace/   -> terrenos e produtos
  producao/      -> gestao de cultivos
  negociacoes/   -> propostas + chat
  assistente/     -> IA agricola
  perfil/        -> conta do utilizador
  admin/         -> painel de administracao
```

ANALOGIA COM JAVA — organização por feature

É como teres um **pacote (package) por módulo de negócio** em vez de um pacote gigante 'controllers' e outro 'models'. Tudo o que diz respeito ao marketplace vive junto — mais fácil de encontrar e manter.

5. O back-end — dados, login e bastidores

Aqui quase não escreveste 'servidor' no sentido clássico. Em vez disso, usaste o **Supabase** (que já traz tudo) e umas **funções pequenas** na Cloudflare. Vamos por partes.

5.1 Base de dados: PostgreSQL (tabelas)

Os dados vivem numa base de dados **relacional** (Postgres): tabelas com colunas, tal como aprendeste em SQL. Exemplos de tabelas do projeto: `profiles` (utilizadores), `properties` (terrenos), `negociacoes`, `mensagens`.

```
-- consultar terrenos, do mais recente para o mais antigo
select * from properties order by created_at desc;
```

ANALOGIA COM JAVA — base de dados

É o mesmo **SQL + JDBC** que já conheces. A diferença: em vez de abrires uma ligação JDBC no servidor, o **próprio browser** faz o pedido através da biblioteca do Supabase — e a segurança é garantida pelas regras RLS (a seguir), não por código Java no meio.

5.2 RLS — segurança dentro da própria base de dados

Como é que o browser pode falar diretamente com a base de dados sem ser um perigo? Graças ao **RLS (Row Level Security)**: regras, escritas na própria base de dados, que dizem **quem pode ver e mudar cada linha**. Por exemplo: 'só as duas partes de uma negociação podem ler as suas mensagens'.

ANALOGIA COM JAVA — RLS

Lembra-te do **Spring Security** a proteger endpoints. O RLS faz o mesmo, mas **dentro da base de dados** e ao nível de cada **linha**. Mesmo que alguém tente pedir dados de outro utilizador, a base de dados recusa. A segurança não depende do front-end.

5.3 Autenticação (login)

O Supabase Auth trata do registo, do login com email/palavra-passe, do login com Google e da recuperação de palavra-passe. Quando o utilizador entra, recebe uma **sessão** (um bilhete temporário) que o identifica nos pedidos seguintes.

5.4 Storage (fotos) e Realtime (tempo real)

- **Storage** — onde ficam as imagens (fotos de terrenos, foto de perfil). Antes de enviar, o browser **comprime** a foto (de ~4 MB para ~250 KB), poupando dados e espaço.
- **Realtime** — o Supabase avisa o browser **na hora** quando algo muda numa tabela. É isto que faz o chat das negociações aparecer ao vivo, sem recarregar a página.

5.5 Cloudflare Functions — esconder os segredos

A IA (Google Gemini) e os pagamentos (PaySuite) usam **chaves secretas**. Se essas chaves fossem para o browser, qualquer pessoa as poderia roubar. Solução: pequenas funções que correm no **servidor** da Cloudflare. O browser pede à função; a função usa a chave secreta e devolve só o resultado.

```
Browser -> /api/ai-chat -> Google Gemini  
      (a chave secreta vive so aqui no meio)
```

ANALOGIA COM JAVA — Cloudflare Functions

Cada ficheiro em `functions/api/` é como um **endpoint REST** (um `@RestController` minúsculo). Recebe um pedido, faz uma coisa, devolve JSON. Existem para guardar segredos e falar com serviços externos.

6. Fluxos completos (juntar tudo)

Para veres as peças a trabalhar juntas, aqui estão dois percursos reais, do clique até aos dados.

6.1 Publicar um terreno com foto

- 1. O utilizador preenche o formulário no **front-end** e escolhe uma foto.
- 2. O browser **comprime** a foto e envia-a para o **Supabase Storage**.
- 3. O Storage devolve um **endereço (URL)** da foto.
- 4. O browser **insere** uma linha na tabela **properties** com os dados e o URL.
- 5. O **RLS** confirma que o utilizador tem direito a inserir; o terreno fica visível no marketplace.

6.2 Perguntar ao assistente de IA

- 1. O utilizador escreve a pergunta no **front-end**.
- 2. O browser envia-a para a função `/api/ai-chat` (Cloudflare).
- 3. A função junta um **prompt de sistema** (as regras do AgroBot) e chama o **Google Gemini** com a chave secreta.
- 4. O Gemini responde; a função devolve o texto ao browser, que o mostra na conversa.

7. Segurança — o essencial

Conceito	Explicação simples
Chave anon (pública)	Vai no browser. Sozinha não dá acesso a nada: o RLS decide tudo.
Chave service_role (secreta)	Poder total. Vive só no servidor (Cloudflare), nunca no browser.
RLS	Regras na base de dados que limitam cada utilizador aos seus dados.
Functions	Escondem as chaves da IA e dos pagamentos.
Números de telefone	São ocultados nas descrições e no chat, por segurança.

ANALOGIA COM JAVA — as duas chaves

É como teres uma credencial **de leitura pública** e uma credencial **de administrador**. A pública pode andar à vista porque não faz mal (as regras protegem tudo); a de administrador nunca sai do servidor.

8. Como o programa vai parar à internet (deploy)

- O código está no **GitHub** (o repositório).
- Sempre que se envia código novo (`git push`), a **Cloudflare** reconstrói e publica o site automaticamente — é o **deploy automático**.
- O **Supabase** aloja a base de dados, o login e as fotos.
- Um pequeno 'ping' periódico mantém o Supabase (plano gratuito) acordado.

ANALOGIA COM JAVA — o deploy

Onde em Java farias `mvn package` e copiarias um `.jar` para um servidor, aqui basta um `git push`: a Cloudflare faz o 'build' e coloca o site no ar, em minutos, sozinha.

9. Glossário rápido — Java para Web

Termo web	Pensa nisto como...
Componente (React)	Uma classe pequena que devolve interface
Props	Os parametros de um metodo
Estado (state)	Um atributo que, ao mudar, redesenha o ecrã
JSX	HTML escrito dentro do código
TypeScript	JavaScript com tipos (mais perto do Java)
npm	Maven/Gradle (gestor de dependencias)
Vite	A ferramenta de build (compilar/empacotar)
Endpoint / Function	Um <code>@RestController</code> pequeno
RLS	Spring Security ao nível da linha da BD
Deploy	Por a aplicação a correr na internet

10. Explicar em 60 segundos

Se te pedirem para resumir de cor, diz algo assim:

"O AgroConecta é uma aplicação web feita em **React com TypeScript** no front-end. Os dados, o login e as fotos ficam no **Supabase** (uma base de dados PostgreSQL gerida, com regras de segurança RLS que protegem cada utilizador). O browser fala diretamente com a base de dados, de forma segura. So a **IA** (Google Gemini) e os **pagamentos** passam por pequenas funções na **Cloudflare**, para esconder as chaves secretas. O site está alojado na Cloudflare Pages e publica-se sozinho a cada envio de código para o GitHub."

Guião gerado para apoio à apresentação do AgroConecta. Front-end: React + TypeScript + Vite + Tailwind. Back-end: Supabase (Postgres, Auth, Storage, Realtime) + Cloudflare Pages Functions. IA: Google Gemini. Pagamentos: PaySuite.